

CO2 ASSESSMENT TOOL 2

CSA1305-TOC

V Dharanireddy 192311219

1. Design the Regular Expression for the Language Represented by the Given Automaton

Given Automaton

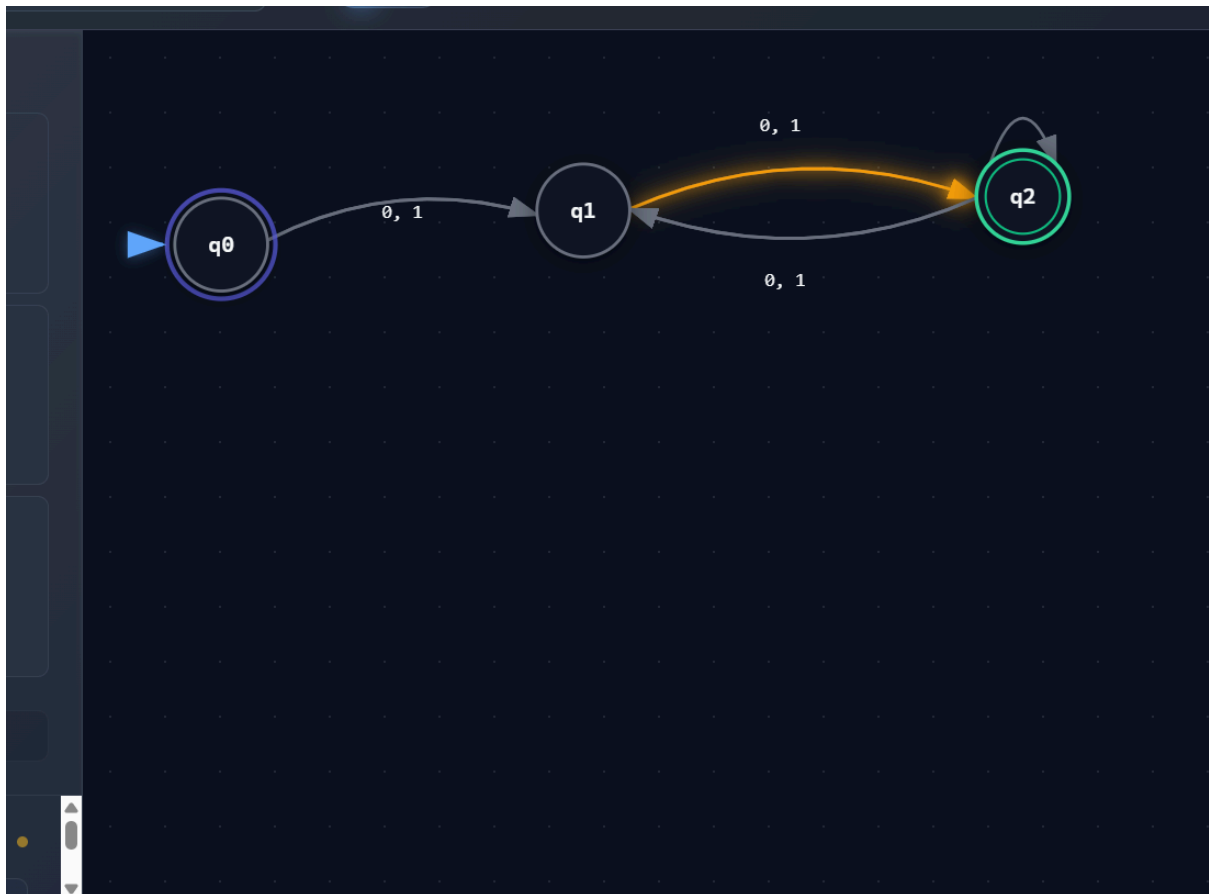
States:

- $q_0 \rightarrow$ Start State
- $q_1 \rightarrow$ Intermediate State
- $q_2 \rightarrow$ Final State

Transitions:

- $q_0 \rightarrow q_1$ on **0,1**
- $q_1 \rightarrow q_2$ on **0,1**
- $q_2 \rightarrow q_1$ on **0,1**

Step 1: Analyze the DFA



To reach the final state:

- One symbol moves from q_0 to q_1 .
- Another symbol moves from q_1 to q_2 .

After reaching q_2 , the automaton alternates between q_2 and q_1 on every input symbol.

Hence accepted strings have **even length (minimum length = 2)**.

Examples Accepted:

00

01

10

11

0011

1100

1010

111100

Rejected:

0

1

000

111

101

Regular Expression

Since every symbol may be either 0 or 1,
the regular expression is

$$((0 + 1)(0 + 1))^+$$

or equivalently,

$$(0 + 1)(0 + 1)((0 + 1)(0 + 1))^*$$

Applications

- Binary protocol validation
- Digital communication
- Network packet verification
- Lexical analysis

Conclusion

The automaton accepts all binary strings having **even length**, represented by

$$((0 + 1)(0 + 1))^+$$

2. Grammar

Given

$$S \rightarrow aSc \mid T \mid U \mid \varepsilon$$

$$T \rightarrow bTc \mid \varepsilon$$

$$U \rightarrow aUb \mid \varepsilon$$

(A) Language Defined

Production 1

$S \rightarrow aSc$

Generates

$a^n c^n \setminus$

Production 2

$T \rightarrow bTc$

Generates

$b^n c^n$

Production 3

$U \rightarrow aUb$

Generates

$a^n b^n$

Hence,

$$L = \{a^n c^n\} \cup \{b^n c^n\} \cup \{a^n b^n\} \cup \{\epsilon\}$$

(B) Derivation Tree for aaabbc

The string

aaabbc

cannot be produced.

Reason:

The grammar produces

$a^n c^n$

or

$b^n c^n$

or

$a^n b^n$

but never

aaa bb c

Therefore

No derivation tree exists.

(C) Ambiguity

Definition:

A grammar is ambiguous if one string has more than one parse tree.

Here every production generates a different language.

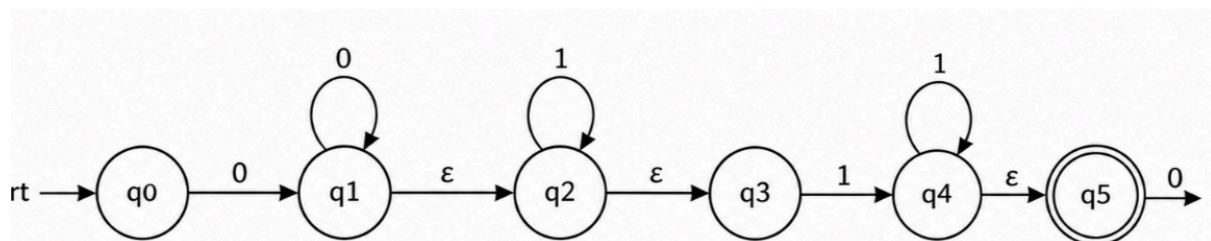
No string has two different parse trees.

Therefore

Grammar is NOT Ambiguous

3. Design an ϵ -NFA for

$00^*1^*11^*0$



Regular Expression

$00^*1^*11^*0$

Simplified

$0+1+0$

ϵ -NFA Construction

States

$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4$

Transitions

$q_0 \xrightarrow{0} q_1$

$q_1 \xrightarrow{0} q_1$

$q_1 \xrightarrow{\epsilon} q_2$

$q_2 \xrightarrow{1} q_2$

$q_2 \xrightarrow{1} q_3$

$q_3 \xrightarrow{1} q_3$

$q_3 \xrightarrow{0} q_4$

Final State

q_4

Examples

Accepted

010

0010

0001110

0011110

Rejected

100

1110

000

Applications

- Compiler lexical analyzer
- Pattern matching
- Intrusion detection

4(i) Show that if L_1 and L_2 are Regular then (L_1+L_2) is Regular

Closure Property

Regular languages are closed under Union.

Suppose

L_1

is recognized by DFA M_1

and

L_2

is recognized by DFA M_2 .

Construct the product DFA

$$Q = Q_1 \times Q_2$$

Accept if either DFA accepts.

Therefore

$$L_1 \cup L_2 \text{ is Regular}$$

Applications

- Spam filtering
- Firewalls
- Search engines
- Compiler scanners

4(ii) Is

$$\{a^p \mid p \text{ is prime}\}$$

Regular?

Answer

No.

Pumping Lemma Proof

Assume

L

is Regular.

Take

$$w = a^q$$

where

q

is prime and larger than pumping length.

Split

$$w = xyz$$

where

$$|xy| \leq p$$

Hence

$$y = a^k$$

Pump

$$i = 2$$

Length becomes

$$q + k$$

(or by choosing an appropriate pumping count, a composite length is obtained), which is not guaranteed to remain prime.

Contradiction.

Therefore

$$\{a^p \mid p \text{ prime}\} \text{ is NOT Regular}$$

5. DFA Minimization

Step 1

Separate

Final

$$\{q_1, q_2, q_4\}$$

Non-final

$$\{q_0, q_3, q_5\}$$

Step 2

Compare transitions.

Equivalent states are merged if every transition leads to equivalent states.

For the given DFA, after partition refinement the equivalent pair is:

$$q_1 \equiv q_4$$

(All other states remain distinguishable.)

Step 3

Merge

q1

and

q4

into one state.

Minimized States

q0

q3

(q1,q4)

q2

q5

Advantages

- Reduced memory
- Lower hardware cost
- Faster execution
- Less power consumption
- Efficient embedded implementation

Applications

- ATM machines
- Smart cards
- IoT devices
- Embedded controllers
- Digital communication systems

Conclusion

The minimized DFA recognizes the **same language** as the original DFA but with fewer states, resulting in simpler hardware and faster execution.